Terraform — Full Lifecycle for Pathnex Infrastructure

$ terraform init

```
Initializing the backend...
Successfully configured the backend "s3"! Terraform will automatically use
this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.67.0

Terraform has been successfully initialized!
```

$ terraform plan

```
Acquiring state lock. This may take a few moments...
Terraform used the selected providers to generate the following execution
plan.
Resource actions are indicated with the following symbols:
  + create
  ~ update in-place
  - destroy

Terraform will perform the following actions:

  # aws_instance.pathnex_application_server will be created
  + resource "aws_instance" "pathnex_application_server" {
      + ami                          = "ami-0c02fb55956c7d316"
      + arn                          = (known after apply)
      + associate_public_ip_address   = true
      + availability_zone             = (known after apply)
      + id                            = (known after apply)
      + instance_type                 = "c5.9xlarge"
      + private_ip                    = (known after apply)
      + public_dns                    = (known after apply)
      + public_ip                     = (known after apply)
      + subnet_id                     = "subnet-07d3a8b91f1234abc"
      + tags                          = {
          + "Name"        = "Pathnex-Application-Server"
          + "Environment" = "Production"
          + "Team"        = "CloudOps"
          + "Project"     = "Pathnex"
          + "Application" = "WebApp"
        }
    }

  # aws_instance.pathnex_db_server will be updated in-place
  ~ resource "aws_instance" "pathnex_db_server" {
        id                    = "i-0db123456789abcd0"
      ~ instance_type          = "r6a.4xlarge" -> "r6a.8xlarge"
        tags                   = {
```

PATHNEX
Shaping Careers, Building Futures

```
            "Name"        = "Pathnex-DB-Server"
            "Environment" = "Production"
            "Team"        = "CloudOps"
            "Project"     = "Pathnex"
            "Application" = "Database"
        }
    }

  # aws_security_group.pathnex_web_sg will be updated in-place
  ~ resource "aws_security_group" "pathnex_web_sg" {
        id                     = "sg-0456789abcdef0123"
      ~ description            = "Pathnex Web SG (old)" -> "Pathnex Web SG
(updated ingress rules)"
        tags                   = {
            "Name"        = "Pathnex-Web-SG"
            "Environment" = "Production"
            "Team"        = "CloudOps"
            "Project"     = "Pathnex"
        }

      ~ ingress {
          ~ cidr_blocks = ["0.0.0.0/0"] -> ["10.0.0.0/16"]
            from_port   = 80
            to_port     = 80
            protocol    = "tcp"
        }
    }

  # aws_instance.pathnex_old_instance will be destroyed
  - resource "aws_instance" "pathnex_old_instance" {
      - ami             = "ami-0c02fb55956c7d316" -> null
      - id              = "i-0old123456789abcd" -> null
      - instance_type   = "t2.micro" -> null
      - private_ip      = "172.31.22.44" -> null
      - public_ip       = "3.82.18.247" -> null
      - tags            = {
          - "Name"        = "Pathnex-Old-Instance"
          - "Environment" = "Staging"
          - "Team"        = "CloudOps"
          - "Project"     = "Pathnex"
          - "Application" = "Legacy"
        } -> null
    }

Plan: 1 to add, 2 to change, 1 to destroy.
Releasing state lock. This may take a few moments...
State lock released.
```

$ terraform apply

```
Acquiring state lock. This may take a few moments...
Terraform used the selected providers to generate the following execution
plan.
Resource actions are indicated with the following symbols:
  + create
  ~ update in-place
  - destroy

Terraform will perform the following actions:
```

PATHNEX
Shaping Careers, Building Futures

```
  # aws_instance.pathnex_application_server will be created
  + resource "aws_instance" "pathnex_application_server" {
      + ami                          = "ami-0c02fb55956c7d316"
      + instance_type                 = "c5.9xlarge"
      + subnet_id                     = "subnet-07d3a8b91f1234abc"
      + tags                          = {
          + "Name"        = "Pathnex-Application-Server"
          + "Environment" = "Production"
          + "Team"        = "CloudOps"
          + "Project"     = "Pathnex"
          + "Application" = "WebApp"
        }
    }

  # aws_instance.pathnex_db_server will be updated in-place
  ~ resource "aws_instance" "pathnex_db_server" {
        id                    = "i-0db123456789abcd0"
      ~ instance_type          = "r6a.4xlarge" -> "r6a.8xlarge"
    }

  # aws_security_group.pathnex_web_sg will be updated in-place
  ~ resource "aws_security_group" "pathnex_web_sg" {
      ~ description           = "Pathnex Web SG (old)" -> "Pathnex Web SG
(updated ingress rules)"
      ~ ingress {
          ~ cidr_blocks = ["0.0.0.0/0"] -> ["10.0.0.0/16"]
        }
    }

  # aws_instance.pathnex_old_instance will be destroyed
  - resource "aws_instance" "pathnex_old_instance" {
      - id              = "i-0old123456789abcd" -> null
      - instance_type   = "t2.micro" -> null
    }

Plan: 1 to add, 2 to change, 1 to destroy.

Do you want to perform these actions?
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.pathnex_old_instance: Destroying... [id=i-0old123456789abcd]
aws_instance.pathnex_old_instance: Destruction complete after 5s

aws_security_group.pathnex_web_sg: Modifying... [id=sg-0456789abcdef0123]
aws_security_group.pathnex_web_sg: Modifications complete after 7s [id=sg-
0456789abcdef0123]

aws_instance.pathnex_db_server: Modifying... [id=i-0db123456789abcd0]
aws_instance.pathnex_db_server: Modifications complete after 11s [id=i-
0db123456789abcd0]

aws_instance.pathnex_application_server: Creating...
aws_instance.pathnex_application_server: Still creating... [15s elapsed]
aws_instance.pathnex_application_server: Creation complete after 23s [id=i-
0123456789abcdef0]

Apply complete! Resources: 1 added, 2 changed, 1 destroyed.
```

PATHNEX
Shaping Careers, Building Futures

```
Outputs:
application_server_public_ip = "3.92.77.188"
db_server_private_ip         = "172.31.55.201"
Releasing state lock. This may take a few moments...
```

$ terraform show

```
terraform show terraform.tfstate


# aws_instance.pathnex_application_server
resource "aws_instance" "pathnex_application_server" {
    ami                  = "ami-0c02fb55956c7d316"
    arn                  = "arn:aws:ec2:us-east-1:123456789012:instance/i-
0123456789abcdef0"
    availability_zone    = "us-east-1a"
    id                   = "i-0123456789abcdef0"
    instance_type        = "c5.9xlarge"
    private_ip           = "172.31.25.120"
    public_ip            = "3.92.77.188"
    subnet_id            = "subnet-07d3a8b91f1234abc"
    tags = {
        "Name"        = "Pathnex-Application-Server"
        "Environment" = "Production"
        "Team"        = "CloudOps"
        "Project"     = "Pathnex"
        "Application" = "WebApp"
    }
}

# aws_instance.pathnex_db_server
resource "aws_instance" "pathnex_db_server" {
    ami                  = "ami-0c02fb55956c7d316"
    arn                  = "arn:aws:ec2:us-east-1:123456789012:instance/i-
0db123456789abcd0"
    availability_zone    = "us-east-1b"
    id                   = "i-0db123456789abcd0"
    instance_type        = "r6a.8xlarge"
    private_ip           = "172.31.55.201"
    tags = {
        "Name"        = "Pathnex-DB-Server"
        "Environment" = "Production"
        "Team"        = "CloudOps"
        "Project"     = "Pathnex"
        "Application" = "Database"
    }
}

# aws_security_group.pathnex_web_sg
resource "aws_security_group" "pathnex_web_sg" {
    description = "Pathnex Web SG (updated ingress rules)"
    egress {
        cidr_blocks = ["0.0.0.0/0"]
        from_port   = 0
        protocol    = "-1"
        to_port     = 0
    }

    ingress {
        cidr_blocks = ["10.0.0.0/16"]
```

PATHNEX
Shaping Careers, Building Futures

```
        from_port   = 80
        protocol    = "tcp"
        to_port     = 80
    }

    id   = "sg-0456789abcdef0123"
    name = "Pathnex-Web-SG"
    tags = {
        "Name"        = "Pathnex-Web-SG"
        "Environment" = "Production"
        "Team"        = "CloudOps"
        "Project"     = "Pathnex"
    }
}

# Outputs:
application_server_public_ip = "3.92.77.188"
db_server_private_ip         = "172.31.55.201"
```

ACTUAL STATE FILE

PATHNEX
Shaping Careers, Building Futures


```
{
  "version": 4,
  "terraform_version": "1.9.5",
  "serial": 12,
  "lineage": "a0b1c2d3-e4f5-6789-0123-abcdef987654",
  "outputs": {
    "application_server_public_ip": {
      "value": "3.92.77.188",
      "type": "string"
    },
    "db_server_private_ip": {
      "value": "172.31.55.201",
      "type": "string"
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "pathnex_application_server",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 1,
          "attributes": {
            "ami": "ami-0c02fb55956c7d316",
            "arn": "arn:aws:ec2:us-east-1:123456789012:instance/i-
0123456789abcdef0",
            "availability_zone": "us-east-1a",
            "id": "i-0123456789abcdef0",
            "instance_type": "c5.9xlarge",
            "private_ip": "172.31.25.120",
            "public_ip": "3.92.77.188",
            "tags": {
              "Name": "Pathnex-Application-Server",
```

```
          "Environment": "Production",
          "Team": "CloudOps",
          "Project": "Pathnex",
          "Application": "WebApp"
        }
      }
    }
  ]
},
{
  "mode": "managed",
  "type": "aws_instance",
  "name": "pathnex_db_server",
  "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
  "instances": [
    {
      "schema_version": 1,
      "attributes": {
        "ami": "ami-0c02fb55956c7d316",
        "id": "i-0db123456789abcd0",
        "instance_type": "r6a.8xlarge",
        "private_ip": "172.31.55.201",
        "tags": {
          "Name": "Pathnex-DB-Server",
          "Environment": "Production",
          "Team": "CloudOps",
          "Project": "Pathnex",
          "Application": "Database"
        }
      }
    }
  ]
},
{
  "mode": "managed",
  "type": "aws_security_group",
  "name": "pathnex_web_sg",
  "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
  "instances": [
    {
      "schema_version": 1,
      "attributes": {
        "description": "Pathnex Web SG (updated ingress rules)",
        "id": "sg-0456789abcdef0123",
        "ingress": [
          {
            "cidr_blocks": ["10.0.0.0/16"],
            "from_port": 80,
            "to_port": 80,
            "protocol": "tcp"
          }
        ],
        "tags": {
          "Name": "Pathnex-Web-SG",
          "Environment": "Production",
          "Team": "CloudOps",
          "Project": "Pathnex"
        }
      }
    }
  ]
```


PATHNEX
Shaping Careers, Building Futures

```
    }
  ]
}
```

✅ Final Summary

Action Type	Resource	Instance Type	Result
+ Create	`aws_instance.pathnex_application_server`	`c5.9xlarge`	✅ Created
~ Change	`aws_instance.pathnex_db_server`	`r6a.4xlarge` → `r6a.8xlarge`	🔄 Modified
~ Change	`aws_security_group.pathnex_web_sg`	Ingress & description updated	🔄 Modified
- Destroy	`aws_instance.pathnex_old_instance`	`t2.micro`	❌ Destroyed

- `terraform show` — displays the **whole current state** (or a saved plan file) in a human-readable format, showing all tracked resources and their attributes.
- `terraform state show <resource>` — shows **only one specific resource** from the state, in detail.

PATHNEX Shaping Careers, Building Futures

🧩 What is `terraform state show <resource>`?

This command lets you inspect **a single resource's state** in your Terraform state file. It displays all the attributes Terraform is tracking for that resource — such as IDs, IPs, tags, etc.

Example: `terraform state show aws_instance.pathnex_application_server`

If you run:

```
terraform state show aws_instance.pathnex_application_server
```

You get output like this (realistic example):

```
# aws_instance.pathnex_application_server:
resource "aws_instance" "pathnex_application_server" {
    ami                          = "ami-0c02fb55956c7d316"
    arn                          = "arn:aws:ec2:us-east-
1:123456789012:instance/i-0123456789abcdef0"
    associate_public_ip_address  = true
    availability_zone            = "us-east-1a"
    id                           = "i-0123456789abcdef0"
    instance_type                = "c5.9xlarge"
```

```
    private_ip                     = "172.31.25.120"
    public_dns                     = "ec2-3-92-77-188.compute-
1.amazonaws.com"
    public_ip                      = "3.92.77.188"
    subnet_id                     = "subnet-0bb1c79de3EXAMPLE"
    tags = {
        "Name"        = "Pathnex-Application-Server"
        "Environment" = "Production"
        "Team"        = "CloudOps"
        "Project"     = "Pathnex"
        "Application" = "WebApp"
    }
    vpc_security_group_ids       = [
        "sg-0a1b2c3d4e5f6g7h8"
    ]
}
```

Summary:

Command	Output
`terraform show`	Shows the **entire state file** or plan
`terraform show <state-file>`	Shows **entire specific state file**
`terraform state show <resource>`	Shows **detailed info of one resource** in the state

PATHNEX
Shaping Careers, Building Futures

Terraform Import Workflow

Step 1 — Create a Minimal Placeholder Resource

Create a minimal `.tf` file with just the required fields (`ami` and `instance_type`) so Terraform has a target:

```
resource "aws_instance" "web" {
  ami           = "ami-placeholder"
  instance_type = "t3.micro"
}
```

Step 2 — Import the Existing EC2 Instance

Import the real EC2 instance into Terraform's state:

```
terraform import aws_instance.web i-0123456789abcdef0
```

- This associates the resource in your `.tf` file with the existing AWS instance.
- Terraform now knows about all the attributes of the instance internally.

Step 3 — Use `terraform state show` to Inspect the Real Resource

Run:

```
terraform state show aws_instance.web
```

- Terraform prints **all current attributes** of the imported instance, including computed fields like `id`, `arn`, `public_ip`, etc.
- **Do not copy this entire output directly** — many fields are read-only and cannot go in your `.tf` file.

Step 4 — Copy Only Manageable Attributes

From the `terraform state show` output, copy **only the attributes you want Terraform to manage**, such as:

- `ami`
- `instance_type`
- `key_name`
- `subnet_id`
- `vpc_security_group_ids`
- `tags`
- `root_block_device` (only user-configurable fields like `volume_size`, `volume_type`, `delete_on_termination`)
- `credit_specification` and `metadata_options` (optional, if you want to manage them)

❌ **Remove everything else**:
`id`, `arn`, `instance_state`, `private_ip`, `public_ip`, `primary_network_interface_id`, computed `metadata_options` defaults, etc.

PATHNEX
Shaping Careers, Building Futures

Step 5 — Paste and Clean

Example cleaned `.tf` block from `terraform state show`:

```
resource "aws_instance" "web" {
  ami                    = "ami-04505e74c0741db8d"
  instance_type          = "t3.micro"
  key_name               = "my-keypair"
  subnet_id              = "subnet-0a1b2c3d4e5f67890"
  vpc_security_group_ids = ["sg-0a12b34cd56ef7890"]

  tags = {
    Name = "web"
    Env  = "prod"
  }

  root_block_device {
```

```
    delete_on_termination = true
    volume_size           = 8
    volume_type           = "gp3"
  }
}
```

Step 6 — Plan and Apply

Check your configuration against the imported resource:

```
terraform plan
terraform apply
```

- Terraform will now **manage only the attributes you included**.
- Computed/read-only fields remain in the state but are not tracked in the code.

✅ Key Takeaways

Using Terraform to Import and Build Code from Existing Resources

- `terraform state show` is **just a reference** to see the current configuration of a resource in your Terraform state.
- You can also use **`terraform plan`** after import to see what Terraform thinks differs between your `.tf` configuration and the actual resource.
- Together, these commands help you **safely copy only the manageable attributes** into your `.tf` code without including computed or read-only fields.

Key Points:

1. **`terraform state show`** → Shows all attributes of the resource in the state (full actual values).
2. **`terraform plan`** → Shows differences between your `.tf` code and the imported resource; highlights what Terraform can manage.
3. **Do not copy everything** from either command — only copy fields you intend Terraform to manage, such as:
    - `ami`
    - `instance_type`
    - `key_name`
    - `subnet_id`
    - `vpc_security_group_ids`
    - `tags`
    - `root_block_device` (user-configurable fields)

Using both **`terraform state show`** and **`terraform plan`** together gives a **complete reference** to build a clean, manageable resource block in your `.tf` code.

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

RESOLVING DRIFT

Step 1 — Detect Drift

Use:

```
terraform plan
```

- Terraform will show a **diff** between the configuration in your `.tf` files and the actual state in the cloud.
- Any **manual changes** in the cloud will appear as **"forces replacement"** or **updates**.
- Example output:

```
~ aws_instance.web
    instance_type: "t2.micro" => "t3.micro"
```

This shows the instance type in the cloud is `t2.micro`, but your config has `t3.micro`.

Step 2 — Decide How to Handle the Drift

PATHNEX
Shaping Careers, Building Futures

You have two options:

Option 1 — Accept the Drift (Update Terraform)

- If the change in the cloud is **intended**, update your `.tf` file to match the cloud.

Example:

```
resource "aws_instance" "web" {
  instance_type = "t2.micro"  # match current cloud
  ami           = "ami-04505e74c0741db8d"
  ...
}
```

- Then run:

```
terraform plan
terraform apply
```

✅ This aligns Terraform config with the actual resource.

Option 2 — Revert the Resource to Terraform Configuration

- If the change in the cloud is **unwanted**, let Terraform restore it:

```
terraform apply
```

- Terraform will update the resource to match your `.tf` configuration.
- This may **modify in-place** or **replace the resource** depending on which attributes changed.

Step 3 — Prevent Future Drift

- Avoid making changes directly in the cloud; use Terraform exclusively.
- Enable **remote state locking** to prevent concurrent changes.
- Track all managed attributes in your `.tf` code.
- Use `terraform plan` regularly to detect drift early.

PATHNEX
Shaping Careers, Building Futures